

Chapter 25: Kotlin/Native

By Victoria Gonda

While Kotlin began its existence focused on the JVM, it has since expanded in scope to include native and web development. In fact, as you'll see in the next chapter "Kotlin Multiplatform", alongside its popularity in Android development, you can even use Kotlin to assist in building iOS apps.

The technology used to bring Kotlin beyond the JVM is called **Kotlin/Native**. Kotlin/Native allows you to compile Kotlin code outside of virtual machines, resulting in self-contained binaries that are native to the environment in which they're run. The Kotlin/Native compiler was announced in 2017, and in late 2018 reached version 1.0.

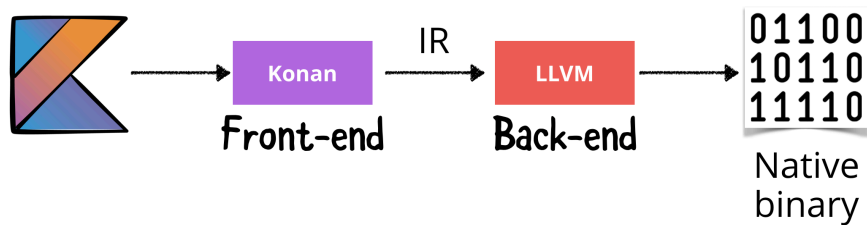
While this chapter looks at **Kotlin/Native** specifically, You can also use **Kotlin/JS** to compile Kotlin code to JavaScript.

Note: If you want to learn more about Kotlin/JS, you can check out the docs at <https://kotlinlang.org/docs/reference/js-overview.html>.

In previous chapters, you've used IntelliJ IDEA to create and run Kotlin code. IntelliJ utilized the version of the Kotlin compiler for the JVM to build and run Java bytecode. In this chapter, you'll see how to install and use the Kotlin/Native compiler outside of IntelliJ in order to create native binaries that can be run outside the JVM.

Konan and LLVM

This diagram below shows the process through which Kotlin/Native takes Kotlin code and turns it into native code.



The Kotlin/Native compiler itself is named **Konan**. Konan handles the **front-end** of the process, compiling the Kotlin source code into an **Intermediate Representation (IR)** that is the input to the **back-end** of the process. Kotlin/Native leverages the **LLVM compiler** for the back-end.

If you're not familiar with LLVM, at one point it stood for "Low-Level Virtual Machine", but it actually has nothing to do with virtual machines as the term is used today. The name is now just a stand-alone acronym.

LLVM was created initially by a team led by Chris Lattner, who also led the team that created the Swift language at Apple. LLVM is a set of components that optimize IR code and compile it to machine-dependent binaries.

By combining Konan with LLVM, Kotlin/Native lets you produce native executables from your Kotlin code. In theory, Kotlin/Native can be used on any platform supported by LLVM.

Installation

You can download v1.4.20 or later of Kotlin/Native from GitHub: <https://github.com/JetBrains/kotlin-native/releases>. There are a number of development builds released there, and you may want to stick to downloading the latest stable version. Distributions are available for macOS, Windows, and Linux.

Note: This chapter uses the macOS compiler. If you're not using macOS, make sure you're downloading the correct compiler for your machine.

For example, go to the download page <https://github.com/JetBrains/kotlin/releases/tag/v1.4.20>. At the bottom of the page, you can download a tarball of Kotlin/Native for macOS: **kotlin-native-macos-1.4.20.tar.gz**.

If you do a web search instead, be careful searching for things like “install kotlin compiler macos” or using **Homebrew** with “brew install kotlin”. That will lead you to installing the same **kotlin-jvm** compiler that you used with SDKMan! in Chapter 24, “Scripting with Kotlin.” to install in Chapter 24, “Scripting with Kotlin.” That version of the Kotlin compiler turns Kotlin source code into Java bytecode, and not the Kotlin/Native compiler Konan uses for making native executables.

You'll see another example of using the kotlin-jvm compiler in this chapter's challenges, but the rest of the chapter itself is focused on using Kotlin/Native.

In a terminal window, navigate to wherever you downloaded the compiler:

```
cd ~/Downloads/
```

Then, untar and unzip the file.

```
tar -xzf kotlin-native-macos-1.4.20.tar.gz
```

Note: If you're directly copying the command from above, make sure the file name is correct for the version you downloaded. You may need to modify the command to match.

Then, move the resulting folder to a location in your home directory, for example, a **bin** folder.

For example:

```
mv kotlin-native-macos-1.4.20 /usr/local/bin
```

Next cd into that folder and take a look at what's inside using `tree -L 2`:

```
cd /usr/local/bin/kotlin-native-macos-1.4.20
tree -L 2
```

Note: If you're on macOS, you may need to install `tree` using `brew install tree`

```
➔ kotlin-native-macos-1.4.20 tree -L 2
.
├── README.md
├── RELEASE_NOTES.md
├── bin
│   ├── cinterop
│   ├── generate-platform
│   ├── jsinterop
│   ├── klib
│   ├── konan-lldb
│   ├── konanc
│   ├── kotlinc
│   ├── kotlinc-native
│   └── run_konan
├── klib
│   ├── cache
│   └── common
├── konan
│   ├── konan.properties
│   ├── lib
│   ├── nativelib
│   ├── platformDef
│   ├── platforms
│   └── targets
├── licenses
│   ├── LICENSE.txt
│   ├── NOTICE.txt
│   └── third_party
├── sources
│   ├── kotlin-stdlib-common-sources.zip
│   ├── kotlin-stdlib-native-sources.zip
│   ├── kotlin-test-annotations-common-sources.zip
│   ├── kotlin-test-common-sources.zip
│   ├── org.jetbrains.kotlinx.kotlinc-cli-common-sources.zip
│   └── org.jetbrains.kotlinx.kotlinc-cli-native-sources.zip
└── tools
    ├── env_blacklist
    └── konan-lldb.py

14 directories, 22 files
```

You see a few different items in the **bin** subfolder including **konanc** and **kotlinc**, the compilers you need for Kotlin/Native code. If you run a `diff` command on the two files, you see that they are in fact identical.

```
diff bin/kotlinc bin/konanc
```

Doing a `less` on either file, you can see that both reference a shell script named **run_konan**:

```
less bin/konanc
```

You'll want to add the **bin** subfolder of the Kotlin/Native distribution into your path by adding an export like this to your **.bashrc** file (or the equivalent for your shell):

```
export PATH="$PATH:/usr/local/bin/kotlin-native-macos-1.4.20/  
bin"
```

Then run a source command on your **.bashrc** file to make the path update take effect in your current terminal session.

```
source ~/.bashrc
```

If you've also installed the `kotlin-jvm` compiler, for example, like in Chapter 24 using SDKMan! or by using Homebrew on macOS, then you need to be careful to make sure you're using the right version of the Kotlin compiler when you want to compile code to native binaries and not to Java bytecode. If you stick to using **konanc** and not **kotlinc**, then you'll be sure to be using the Kotlin/Native version and not the `kotlin-jvm` version.

Another important point to make is that when you use Kotlin/Native with IDEs such as **IntelliJ IDEA** and **Android Studio**, the build plugins you use with **Gradle** will install a separate version of the Kotlin/Native compiler than the one you just installed. So in that case you don't need to associate the downloaded compiler with your IDE in any way; the IDE will use its own version of the compiler, based on settings in the build files.

Hello, Kotlin/Native

With a Kotlin/Native compiler in place, you can now proceed to write your first native program in Kotlin.

You can use IntelliJ IDEA to create and run Kotlin/Native code, or you can use a plain text editor. Creating a Kotlin/Native project in IntelliJ usually uses Gradle as the build system. In this chapter, you're working outside of Gradle and manually compiling the Kotlin code yourself.

Modern text editors will often have plugins for coding in languages like Kotlin, adding features such as syntax highlighting and code completion. For example, **Visual Studio Code** has a “Kotlin Language” extension. That extension does not provide code completion at this time, but does do syntax highlighting.

Create a new file in your text editor of choice and save it as **hello.kt**.

In the file, create a `main` function just like you have in previous chapters for code that ran on the JVM:

```
fun main() {  
}
```

In earlier versions of Kotlin prior to 1.3, you had to supply `main` with an `args` parameter of type `Array<String>`, but now that’s no longer necessary if you aren’t going to use command-line arguments.

Add a single `println` statement to `main()`, and print the string “Hello, Kotlin/Native!”.

```
println("Hello, Kotlin/Native!")
```

Save the file then switch to a terminal window and enter the command to compile the code into a native executable.

```
konanc -o hello hello.kt
```

The `-o` option lets you specify the name of the output executable. The command as written must be run in the same directory as the **hello.kt** file.

Go ahead and run the compiler command. The first time you compile a file with `konanc`, it’s going to take a few minutes, since the Kotlin compiler will download the files it needs to do the compilation, including the LLVM compiler. After your first run of `konanc`, compilations will take much less time.

When the command is done, do an `ls -l` command to see the output file, **hello.kexe**. The **kexe** extension denotes “Kotlin executable”. Typically you’ll want to remove that extension since it’s not a standard Unix convention.

If you don’t use the `-o` option, the default output filename is **program.kexe**.

Next, run `./hello.kexe` and see the native executable run:

```
Hello, Kotlin/Native!
```

The file **hello.kexe** is a native binary for macOS. The file was produced using Konan and LLVM. There is no Java bytecode to be found as far as the native binary is concerned, and you do not need the JVM to run the file.

Kotlin Standard Library

One thing you may notice is that, when compiled with v1.4.20 of Kotlin/Native, the executable file **hello.kexe** is about 945 KB on macOS, which is over 100 times larger than the executable for an equivalent program written in C would be. That large size is because the **Kotlin standard library** is statically linked in to the executable file.

This means that you're free to use anything in the standard library in your program.

You can see that by adding something like a list into the program using the standard library `listOf()` function.

```
val numbers = listOf(1, 2, 3)
```

Then you can print out the size of the list.

```
println(numbers.size)
```

Go ahead and add that code and recompile the executable using the same command as before:

```
konanc -o hello hello.kt
```

Then run the executable file again. You'll see the size of the small list printed to the console.

Notice that, when including the standard library call to `listOf()`, you don't need to do any imports or do any special link command when you re-compile the file.

Challenges

1. Create a Kotlin/Native program similar to the first one you wrote, but this time with some more standard library code. In particular, try printing out the squares of all even integers less than or equal to 100.

As a hint, you can use the `map` and `filter` functions to do this. Also, you can use a range operator and the `toList()` function to make a list of numbers:

```
val numbers = (1..100).toList()
```

2. Compile the code from the last challenge with the `kotlin-jvm` compiler instead of Kotlin/Native. Remember that the `kotlinc` here needs to be different than the version that comes with Kotlin/Native. You can get this one via Homebrew using `brew install kotlin`.

Watch out on setting up your path correctly so that you know which version of the compiler you are using, Kotlin/Native or `kotlin-jvm`. Homebrew will typically install in `/usr/local/bin`, so you can use that full path in order to make sure you're using the `kotlin-jvm` version.

A hint here is that you can compile using:

```
/usr/local/bin/kotlinc evensquares.kt
```

This will produce Java bytecode in a file named **EvenSquaresKt.class**. You can then run that bytecode on the JVM using:

```
/usr/local/bin/kotlin EvensquaresKt
```

This is similar to the process you would use when compiling Java code with the `javac` compiler and running with the `java` command.

Key points

- **Kotlin/Native** is used to compile Kotlin code to native binaries that can be run without virtual machines and Kotlin runtimes.
- The name of the Kotlin/Native compiler is **Konan**. The command to run Konan is `konanc`.
- Kotlin/Native leverages the **LLVM** compiler to produce native binaries. Konan acts as a front-end for LLVM, producing an Intermediate Representation for LLVM.
- The Kotlin/Native compiler can be installed from its GitHub page at <https://github.com/JetBrains/kotlin-native/releases>.
- When installing the Kotlin/Native compiler, be sure to distinguish it from the `kotlin-jvm` compiler used to create Java bytecode.
- Kotlin/Native code starts with a `main` function, similar to other C-like languages.
- The Kotlin standard library is statically linked to Kotlin/Native executables.

Where to go from here?

Now that you've got a handle on the use of Kotlin/Native and what it's for, the next chapter will utilize Kotlin/Native in the context of a Kotlin Multiplatform project. Kotlin/Native will be used by your IDE to compile shared Kotlin code for use on iOS.

If you want to dig deeper into Kotlin/Native, a good place to start is the Kotlin/Native page on the Kotlin language web site: <https://kotlinlang.org/docs/reference/native-overview.html>.